

Data Transer Protocol documentation

Add your content using `reStructuredText` syntax. See the [reStructuredText](#) documentation for details.

Network Simulator

Table of Contents

- [Packet](#)
- [Server](#)
- [Client](#)
- [Network Sim](#)

[Packet](#)

`class packet.Packet` [\[source\]](#)

Bases: **object**

HEADER_FORMAT = 'i i i 16s'

HEADER_SIZE = 28

MAX_PAYLOAD = 1024

checksum() [\[source\]](#)

Creates md5 digest which can be used to detect if bits are corrupted

params:

data: current set of data being transferred

create_packet(ack_num, flags, payload=b'') [\[source\]](#)

Creates packets that are transferred between the client and server

params:

seq_num: current sequence number ack_num: current ACK number flags: flags triggered during transfer

payload: data being sent in current packet

unpack() [\[source\]](#)

unpacks raw bytes being transferred

params:

raw_bytes: raw bytes of data currently in packet

[Server](#)

`class server.Server(server_port, target_ip, target_port, timeout)` [\[source\]](#)

Bases: **object**

finish_transfer() [\[source\]](#)

Closes connection with client when transfer is finished

send_data(payload) [\[source\]](#)

Sends data payload to the client

parma:

payload: current chunk of data being sent to the client

start_timer() [\[source\]](#)

stope_timer() [\[source\]](#)

timeout_action() [\[source\]](#)

wait_for_acks() [\[source\]](#)

listens for ACKS being sent from the client

Client

class client.**Client**(*client_port*, *server_ip*, *server_port*, *done_event*) [\[source\]](#)

Bases: **object**

listen() [\[source\]](#)

Function for the client to listen for and receive data from the server

send_ack(ack_num) [\[source\]](#)

Constucts and sends ACK to server

params:

ack_num: number of ack being sent to server

Network Sim

network_sim.**network_sim**(*client_port*, *target_ip*, *target_port*, *loss*, *corrupt*, *reorder*) [\[source\]](#)

Simulates a network connection between a client and server. Includes data loss check and corrupted data check

params:

client_port: port of the connected client target_ip: ip of the of the target server target_port: port of the target server loss: chance for a simulated data loss to be triggerd corrupt: chance for a simulated data corruption to occur reorder: chance for a simulated reorder to occur